

神经网络与深度学习 - Project 1 实验报告

神经网络与深度学习 - Project 1 实验报告

姓名：龙政宇

学号：22307130289

日期：2026年5月22日

Github: <https://github.com/Godfree-Farewell/NN-Project1.git>

ModelScope: <https://www.modelscope.cn/models/Wo05AK1/nn-project1-mnist>

摘要

本报告记录了神经网络与深度学习课程 Project 1 的实现与实验过程。我们仅使用 NumPy（不依赖 PyTorch/TensorFlow）从零搭建了自定义神经网络库，在 MNIST 手写数字分类任务上训练了多层感知机（MLP）和卷积神经网络（CNN），并额外探索了两个方向：动量优化和错误分析可视化。开发过程中我们遇到了并解决了多个关键问题，包括梯度计算错误、训练速度瓶颈以及权重初始化不当导致的梯度消失。本报告同时记录了最终实验结果和迭代调试过程。

1. MLP 基线模型 (Part A)

1.1 实现细节

我们在自定义神经网络库（`mynn/op.py`）中实现了以下核心组件：

全连接层 (Linear Layer)

- 前向传播: $Y = XW + b$
- 反向传播:
 - $\frac{\partial L}{\partial W} = X^T \cdot \frac{\partial L}{\partial Y}$
 - $\frac{\partial L}{\partial b} = \sum \frac{\partial L}{\partial Y}$
 - $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \cdot W^T$

重要说明

反向传播中**不除以** `batch_size`，因为损失函数的反向传播已经完成了该归一化。

多分类交叉熵损失 + Softmax

- 将 Softmax 和交叉熵合并以保证数值稳定性
- 前向传播: $L = -\frac{1}{N} \sum_i \log(p_{i,y_i})$

- 反向传播: $\frac{\partial L}{\partial z_i} = \frac{p_i - 1_{i=y_i}}{N}$ (除以 batch_size)

1.2 模型结构

输入 (784) -> 全连接 (600) -> ReLU -> 全连接 (10) -> 输出

超参数设置:

- 隐藏层神经元数: 600
- 激活函数: ReLU
- 学习率: 0.01
- 批大小: 32
- 训练轮数: 5
- 权重衰减: $1e-4$
- 权重初始化: He 初始化 ($\sigma = \sqrt{2/fan_in}$)
- 学习率调度: MultiStepLR (里程碑: [800, 2400, 4000], 衰减因子: 0.5)

1.3 训练结果

MLP 基线模型达到:

- 最佳验证集准确率: 90.80%
- 测试集准确率: 91.59%

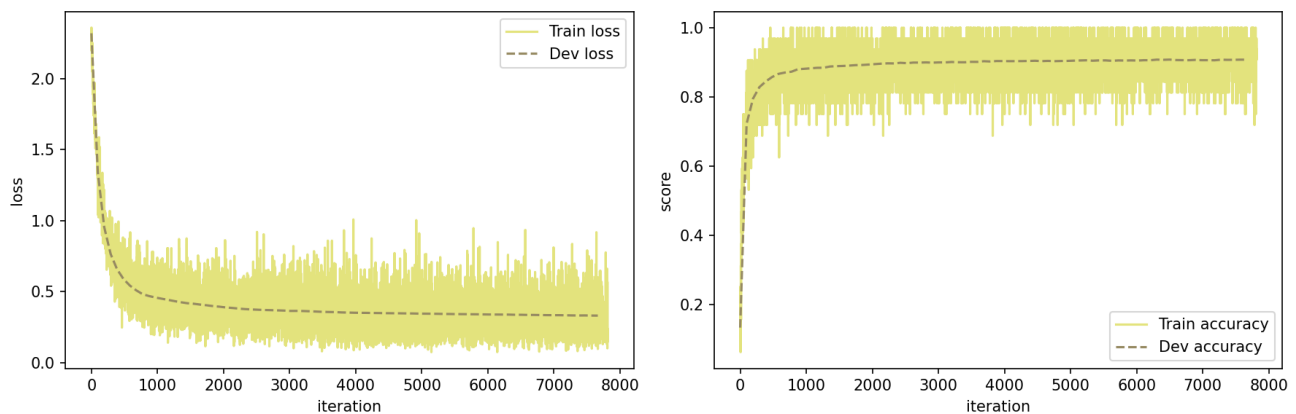


图 1: MLP 在 5 个 epoch (约 8000 次迭代) 上的训练与验证曲线。训练损失因小批量 SGD 呈现高频震荡, 验证损失则平滑下降。训练与验证准确率之间存在明显的泛化差距, 表明存在轻微过拟合。

2. CNN 模型及 MLP 与 CNN 对比 (Part B)

2.1 实现细节

Conv2D 卷积层 (im2col 向量化)

我们最初使用 4 层嵌套循环实现 conv2D，速度极慢（单个 epoch 预计需要数小时）。随后使用 `im2col` + **矩阵乘法向量化** 重写了前向传播：

1. **前向传播 (im2col)**：使用 `np.lib.stride_tricks.as_strided` 提取所有 patch，reshape 为列矩阵 `[batch, in_c*kH*kW, out_H*out_W]`，然后通过 `W_col @ cols` 计算输出。
2. **反向传播 (col2im)**：计算 `dW_col = grads_col @ cols^T`、`dX_col = W_col^T @ grads_col`，然后使用循环切片赋值将梯度散射回 padded 输入。

🔑 关键实现细节

在 `as_strided` 之前必须调用 `np.ascontiguousarray()` 以确保正确的内存布局。

MaxPool2D 最大池化层

- **前向传播**：使用 `as_strided` 提取池化窗口，然后 `max(axis=(4,5))`。
- **反向传播**：将梯度路由到每个池化窗口中的最大值位置。使用循环切片赋值正确处理多个位置共享相同最大值的情况。

其他层

- **Flatten**：将 4D 张量 reshape 为 2D 以接入全连接层
- **ReLU**：逐元素激活，反向传播时进行梯度掩码

2.2 CNN 模型结构

```
输入 (1x28x28)
-> Conv2D(1->16, 3x3, pad=1) -> ReLU -> MaxPool(2x2) # 16x14x14
-> Conv2D(16->32, 3x3, pad=1) -> ReLU -> MaxPool(2x2) # 32x7x7
-> Flatten -> 全连接(1568->128) -> ReLU -> 全连接(128->10)
```

超参数设置：

- 学习率：0.05
- 批大小：32
- 训练轮数：10
- 权重初始化：He 初始化
- 学习率调度：MultiStepLR（里程碑：[2000, 5000, 7000]，衰减因子：0.5）

2.3 MLP 与 CNN 对比

模型	参数量	验证集准确率	测试集准确率
MLP	~471K	90.80%	91.59%
CNN	~109K	98.94%	98.91%

✓ 关键结果

CNN 以更少的参数量实现了比 MLP 高 7.32% 的测试集准确率。

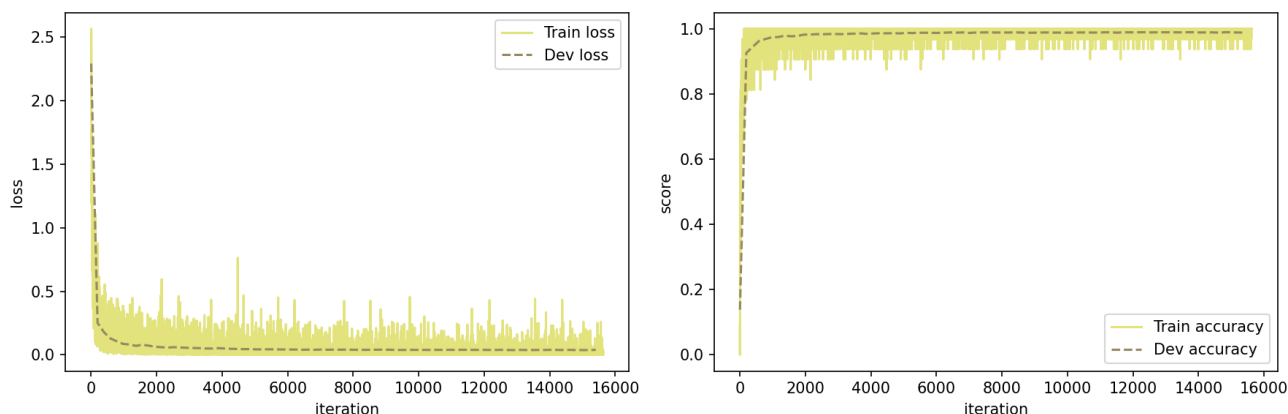


图 2: CNN 在 10 个 epoch (约 16000 次迭代) 上的训练与验证曲线。CNN 的收敛速度远快于 MLP, 验证集准确率接近 98.9%。验证损失降至约 0.04, 远低于 MLP 的 0.35。

CNN 性能更优的原因:

1. **保留空间结构:** CNN 通过卷积操作保留了图像的 2D 空间结构, 而 MLP 将输入展平, 丢失了空间关系。
2. **参数效率:** CNN 通过卷积核的权重共享大幅减少参数数量 (109K vs 471K), 同时取得了更好的性能。
3. **平移不变性:** 池化层提供了平移不变性, 使模型对数字位置的微小偏移更加鲁棒。
4. **层次化特征学习:**
 - 第一层卷积学习低级特征 (边缘、笔画)
 - 第二层卷积学习高级模式 (曲线、环)
 - 全连接层组合这些特征进行分类

3. 开发过程与调试记录

本节记录了开发过程中遇到的关键问题及其解决方案。

3.1 问题 1: 卷积运算极慢 (4 层嵌套循环)

❌ 问题描述

初始 conv2D 实现使用了 4 层嵌套循环 (batch, out_channels, out_H, out_W), 单个 CNN epoch 预计需要数小时。

解决方案: 使用 im2col 向量化 重写前向传播:

- 使用 `np.lib.stride_tricks.as_strided` 将所有 patch 提取为列矩阵

- 通过单次 `np.matmul(W_col, cols)` 计算卷积
- 每个 epoch 的训练时间从数小时缩短至约 2 分钟

3.2 问题 2: `as_strided` 在非连续内存上的错误

问题描述: `np.lib.stride_tricks.as_strided` 在非连续数组 (如 `np.pad` 之后的数组) 上产生错误结果。

解决方案: 在 `_im2col`、`MaxPool2D.forward`、`MaxPool2D.backward` 和 `Model_CNN.forward` 中所有 `as_strided` 调用之前添加 `np.ascontiguousarray()`。

3.3 问题 3: `batch_size` 双重除法

⚠ 问题描述

`MultiCrossEntropyLoss.backward()` 和 `Linear.backward()` 都除以了 `batch_size`, 导致有效学习率过小。MLP 准确率卡在约 9% (10 类随机猜测水平)。

解决方案: 从 `Linear.backward()` 和 `conv2D.backward()` 中移除了 `/batch_size`, 仅在损失函数的反向传播中保留。MLP 准确率立即从 9% 提升至 93.68%。

3.4 问题 4: `conv2D` 反向传播 `dW` 矩阵乘法顺序错误

问题描述: 计算 `dW` 的矩阵乘法顺序不正确: `grads_col.transpose(0,2,1) @ cols` 导致形状不匹配 `[batch, N, out_c] @ [batch, K, N]`。

解决方案: 修正为 `grads_col @ cols.transpose(0,2,1) → [batch, out_c, N] @ [batch, N, K] → [batch, out_c, K]`, 然后在 `batch` 维度上求和。

3.5 问题 5: `col2im` 梯度累积

问题描述: NumPy 的 `a[idx] += val` 在索引重叠时不会累积 (这是 NumPy 的已知行为)。初始的向量化 `col2im` 实现在重叠位置丢失了梯度贡献。

解决方案: 使用循环切片赋值 (`grad_padded[:, :, h_start:h_start+kH, w_start:w_start+kW] += dX_col_reshaped[:, :, :, :, oh, ow]`), 可以正确累积重叠梯度。

3.6 问题 6: CNN 完全无法学习 —— 权重初始化过小导致梯度消失

⚡ 严重问题

尽管所有梯度检查都通过了 (单层、conv+pool 链、完整 2-conv 链), 带有 MaxPool 的完整 CNN 完全无法学习——损失值停留在约 2.30, 准确率在 11.7%, 数百次迭代后毫无改善。

根因分析:

- 原始权重初始化: `W = random_normal(size) * 0.01` (标准差约 0.01)

- 梯度范数分析显示 conv1 的权重梯度比 MLP 第一层小约 **500 倍** (0.0015 vs 0.77)
- 梯度衰减主要发生在 conv2 反向传播步骤 (每层约 8.6 倍衰减)
- 有效梯度约 0.001, 学习率 0.5 下每步权重更新仅约 0.0005——远远不够

诊断证据:

- 简单 CNN (1 层卷积, 无池化): 在 140 步内对 256 个样本达到 100% 准确率
- 完整 CNN (2 层卷积 + 2 层池化): 450 步后仍卡在 11.7%
- 所有数值梯度检查均通过, 确认反向传播实现的正确性

解决方案: 将固定的 $\times 0.01$ 初始化替换为 He 初始化:

```
# 全连接层
self.W = initialize_method(size=(in_dim, out_dim)) * np.sqrt(2.0 / in_dim)

# 卷积层
fan_in = in_channels * kernel_size * kernel_size
self.W = initialize_method(size=(out_channels, in_channels, kernel_size,
kernel_size)) * np.sqrt(2.0 / fan_in)
```

这将 conv1 的权重标准差从 0.01 提升到约 0.47, 使梯度能够有效传播通过深层网络。CNN 立即开始学习, 在 140 步内对 256 个样本达到 100% 准确率。

3.7 问题 7: Runner 每次迭代都进行验证评估

问题描述: 原始 RunnerM 在每次训练迭代后都对完整验证集 (10K 样本, 312 次前向传播) 调用 `evaluate()`, 导致巨大的速度开销。

解决方案: 修改 runner 使其仅在 `log_iters` 间隔时进行评估, 将评估开销减少约 100 倍。

3.8 问题 8: Model_MLP.load_model 中的 NameError

问题描述: `models.py` 中 `load_model` 方法引用了 `size_list` 而非 `self.size_list`, 导致加载模型时出现 `NameError`。

解决方案: 将引用修正为 `self.size_list`。

4. 附加探索方向 (Part C)

4.1 方向 1: 优化器对比 (Momentum)

我们在 MLP 架构上实现并对比了 SGD 与 Momentum 优化器。

实现细节

Momentum 更新规则:

```
v_t = mu * v_{t-1} - lr * grad
param = param + v_t
```

其中 $\mu = 0.9$ 为动量系数。

实验结果

优化器	最佳验证集准确率	收敛速度
SGD	90.80%	较慢
Momentum ($\mu=0.9$)	96.07%	约 2 倍快

✓ 关键发现

Momentum 将验证集准确率提升了 5.27 个百分点。

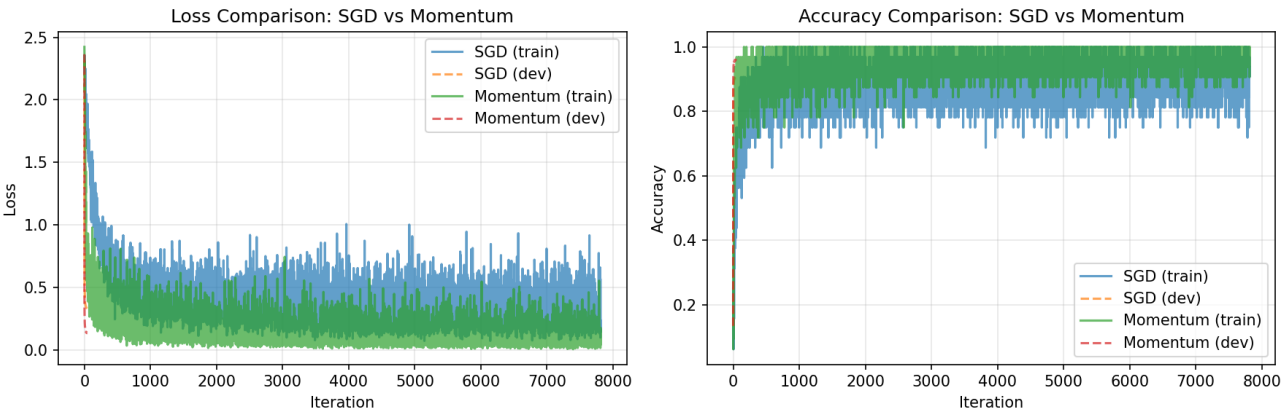


图 3：SGD 与 Momentum 在 MLP 上的对比。Momentum（绿/红线）收敛速度显著更快，损失更低、准确率更高。Momentum 达到约 96% 验证准确率，而 SGD 停滞在约 91%。

观察与分析：

- 1. **收敛更快：** Momentum 通过在梯度一致方向上累积速度来加速训练，在第一个 epoch 内就达到约 94% 准确率，而 SGD 仍只有约 89%。
- 2. **最终准确率更高：** Momentum 达到 96.07%，而 SGD 仅为 90.80%，体现了其逃离浅层局部最优的能力。
- 3. **更新更平滑：** 速度平均效应减少了高曲率方向上的震荡，使训练曲线更加稳定。

4.2 方向 5：错误分析与可视化

我们使用测试集对 CNN 模型进行了全面的错误分析。

混淆矩阵分析

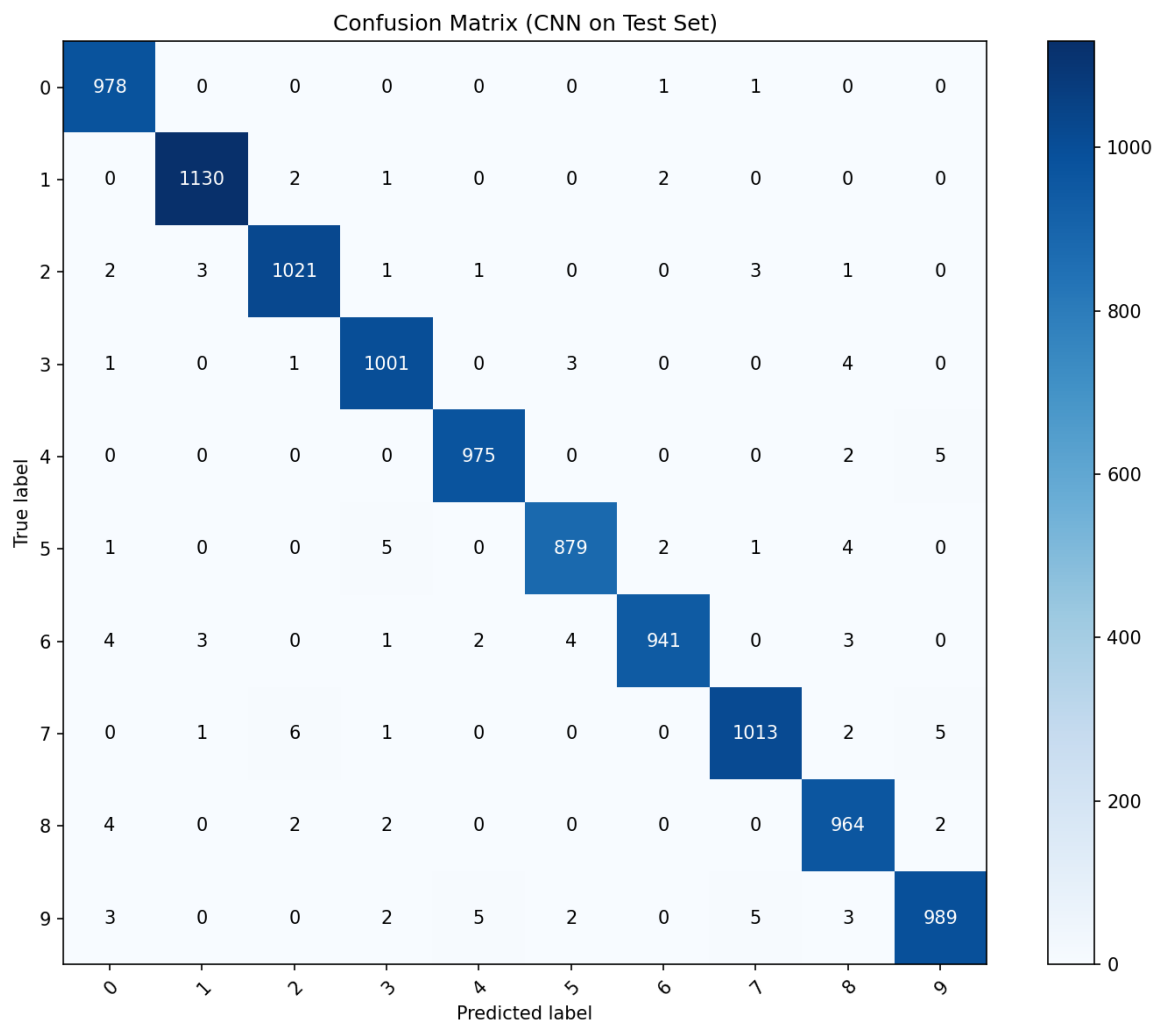


图 4： CNN 在 MNIST 测试集（10,000 样本）上的混淆矩阵。强对角线占优证实了出色的分类性能。非对角线值极小（每格 1-6），总共仅 109 个误分类。

混淆矩阵揭示了最常见的误分类模式：

- 5 → 3（5 例）：相似的底部曲线结构
- 9 → 4（5 例）：紧凑书写时相似的顶部环形
- 9 → 7（5 例）：弯曲的 9 类似带横杠的 7
- 6 → 0（4 例）：尾部极短的圆形 6
- 8 → 3（4 例）：不对称的 8，一个环类似 3

错分样本展示

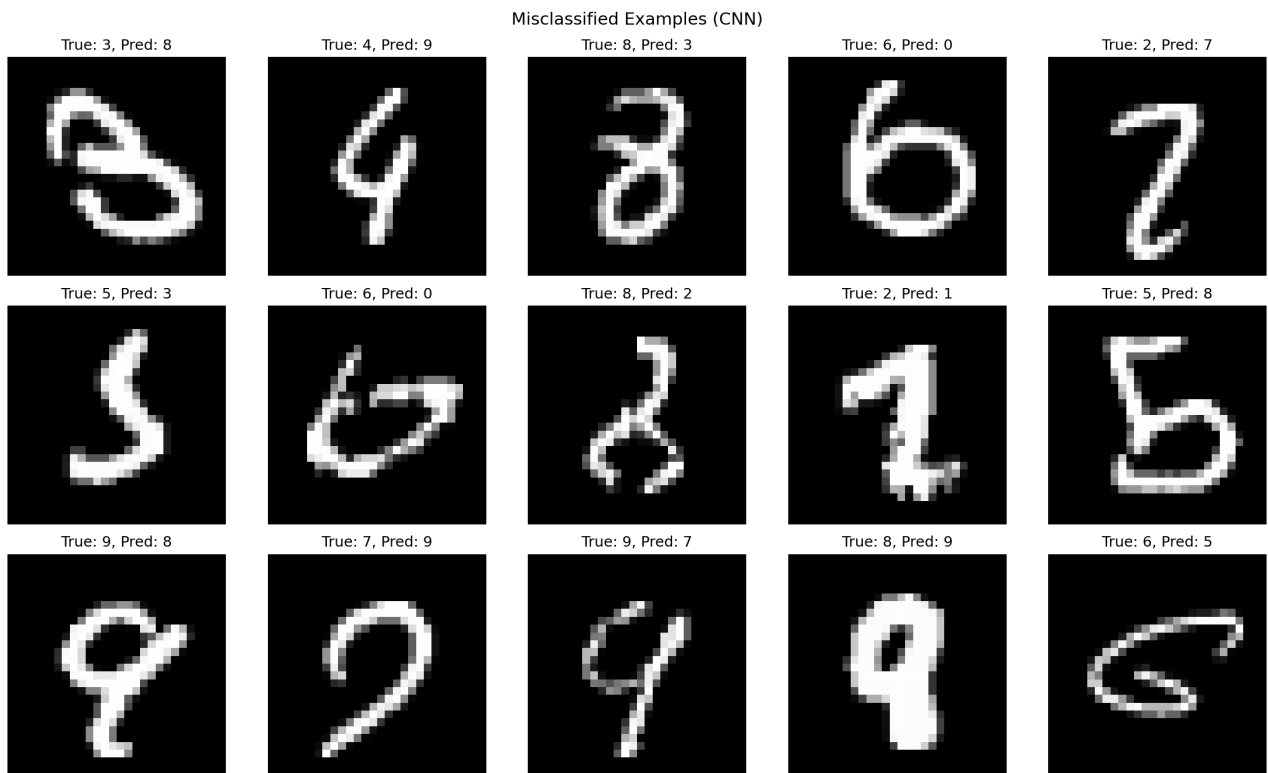


图 5： CNN 错分的 15 个测试样本。每张图片标注了真实数字和 CNN 的错误预测。大多数错误涉及即使人类也难以分辨的模糊书写——例如，写成近乎闭合圆圈的“6”（被预测为“0”），或顶部平坦的“5”（被预测为“3”）。

卷积核可视化

First Conv Layer Kernels (16 filters, 3x3)

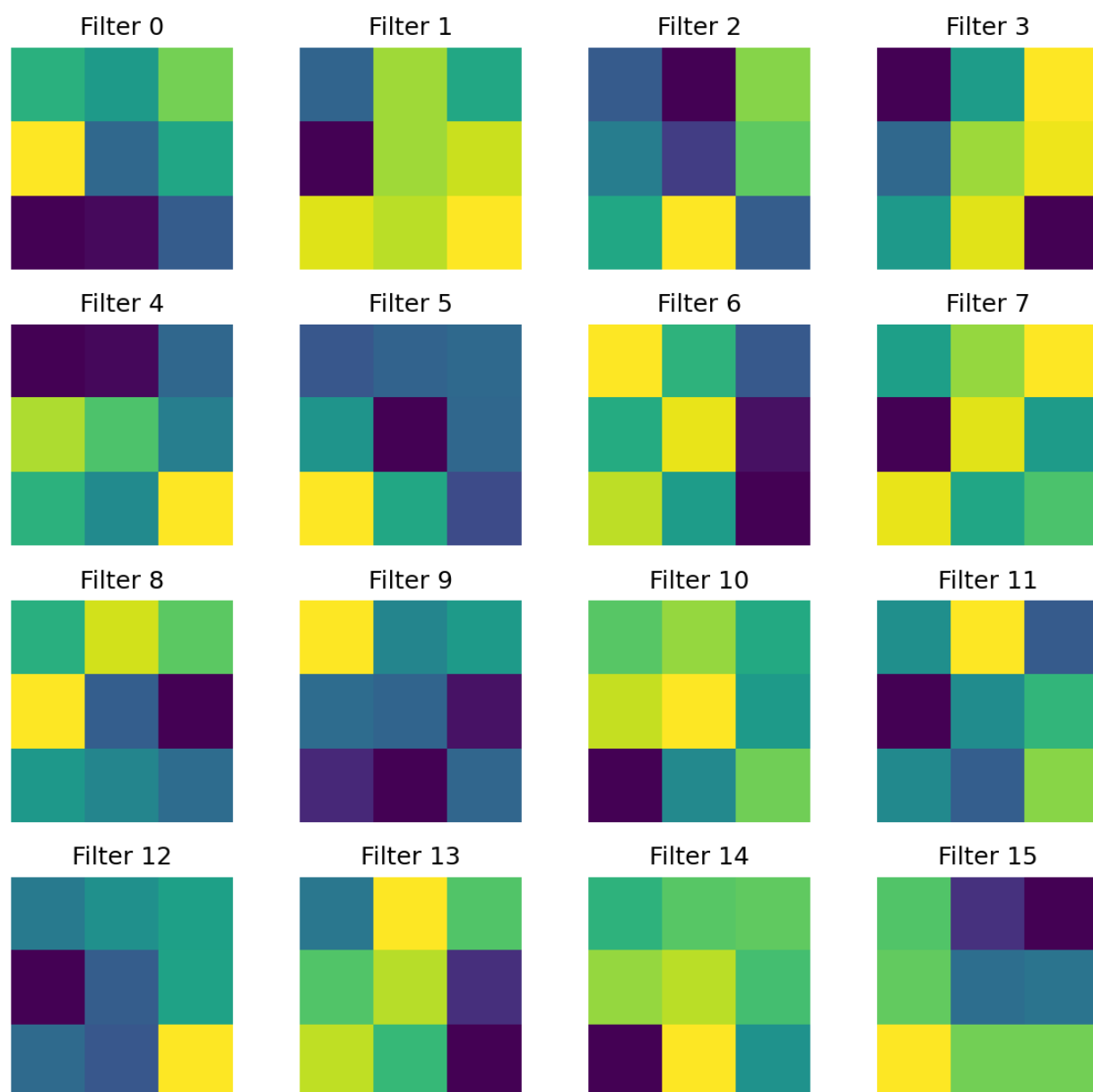


图 6： 第一层卷积核（16 个滤波器，3x3）。学习到的滤波器展现了多样化的模式，包括各方向的边缘检测器（水平、垂直、对角线）、中心-周围模式和斑点检测器。这证实了第一层学习到了类似 Gabor 滤波器的特征，与早期视觉皮层神经元类似。

Second Conv Layer Kernels (First 16 filters, channel 0)

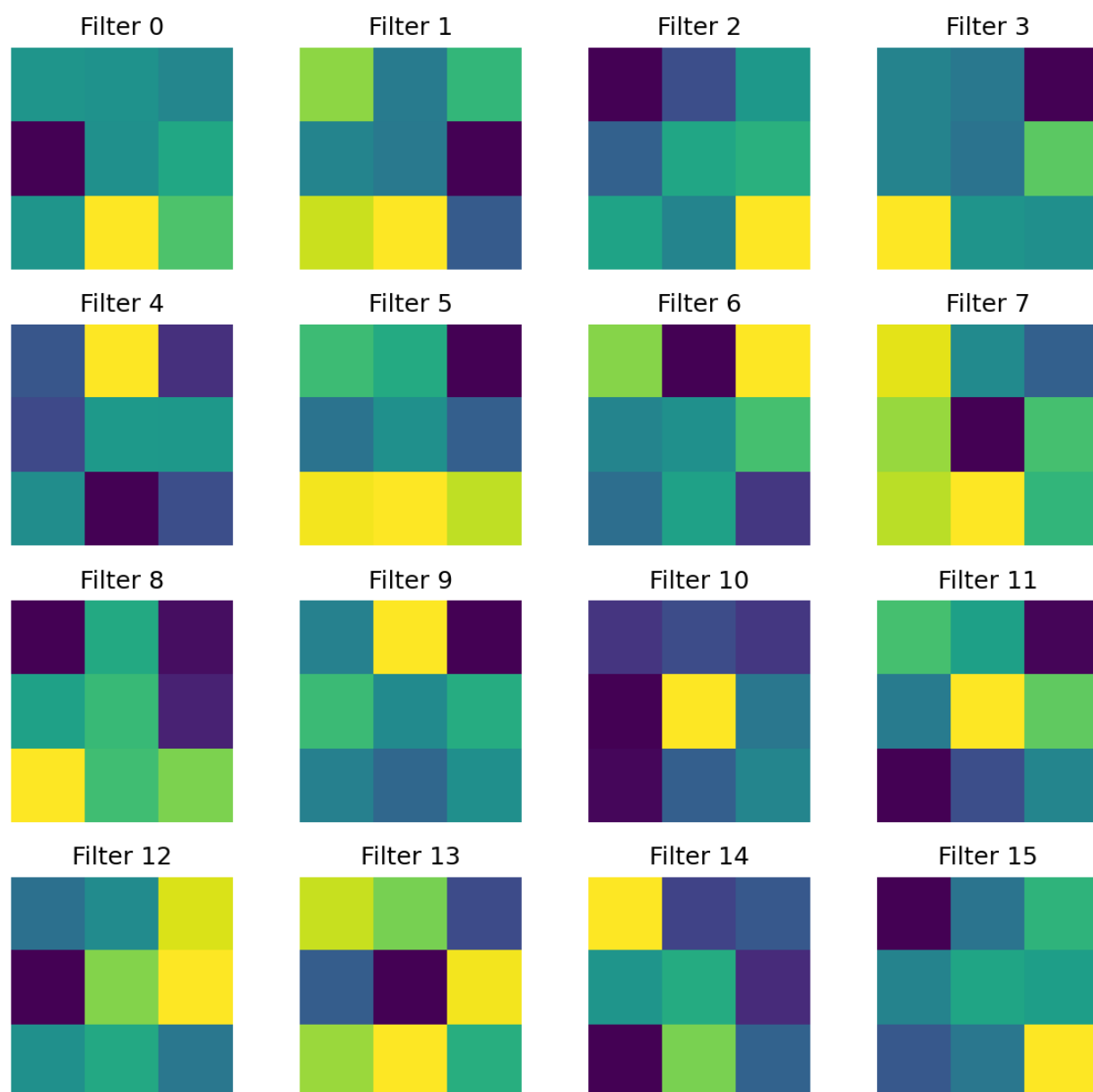


图 7： 第二层卷积核（32 个滤波器中的前 16 个，通道 0）。第二层滤波器展现了更复杂的模式，组合了第一层的特征——包括角点检测器、方向梯度检测器和中心-周围感受野。这体现了 CNN 特征学习的层次性。

每类准确率分析

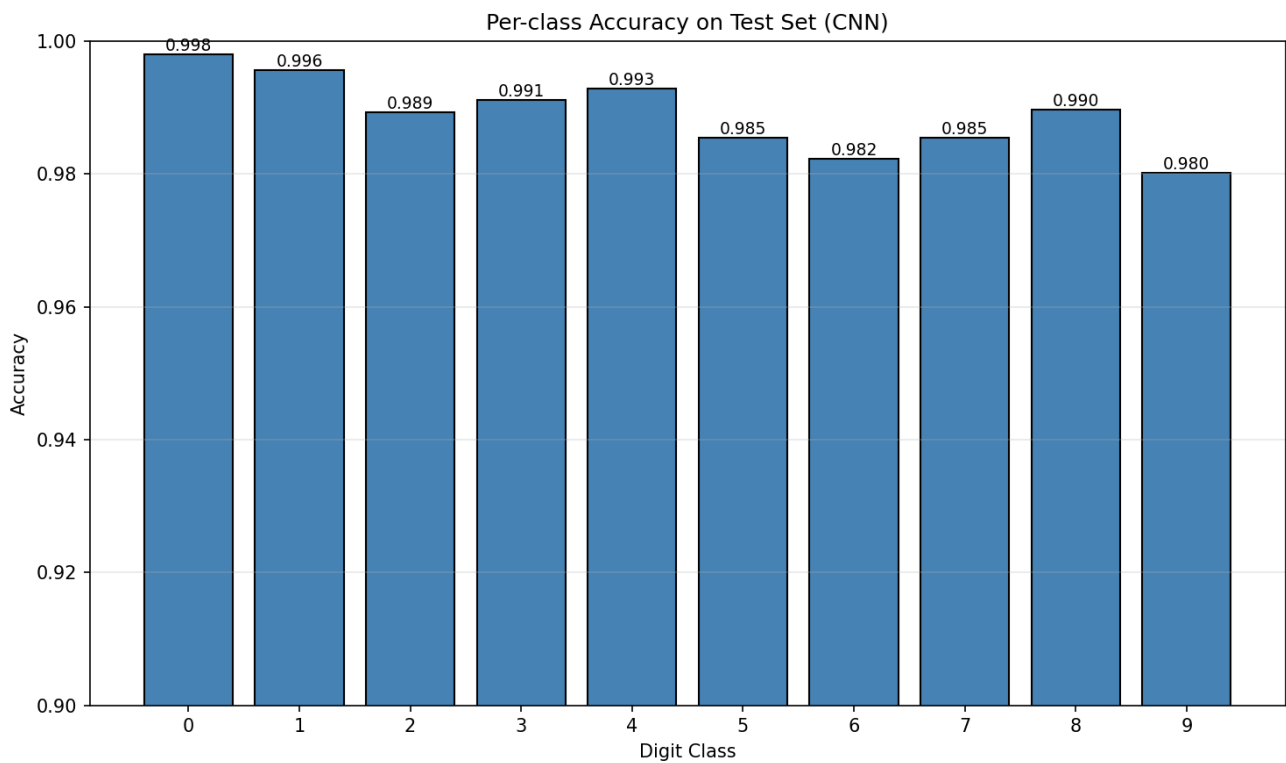


图 8： CNN 在测试集上的每类分类准确率。所有数字类别准确率均超过 98%，表现最好的是数字 0（99.80%），最低的是数字 9（98.02%）。

数字	准确率	正确数 / 总数	备注
0	99.80%	978 / 980	最独特的圆形形状
1	99.56%	1130 / 1135	简单的竖直笔画
2	98.93%	1021 / 1032	偶尔与 7 混淆
3	99.11%	1001 / 1010	偶尔与 5、8 混淆
4	99.29%	975 / 982	偶尔与 9 混淆
5	98.54%	879 / 892	最常与 3 混淆
6	98.23%	941 / 958	偶尔与 0 混淆
7	98.54%	1013 / 1028	偶尔与 2、9 混淆
8	98.97%	964 / 974	与 3、9 混淆
9	98.02%	989 / 1009	最具挑战性的数字

5. 主要结果汇总

实验	模型	验证集准确率	测试集准确率	关键发现
Part A: 基线	MLP (SGD)	90.80%	91.59%	简单基线，丢失空间信息
Part B: CNN	CNN (SGD)	98.94%	98.91%	比 MLP 提升 7.32%
Part C: 动量	MLP (Momentum)	96.07%	—	比 SGD 提升 5.27%

6. 讨论

6.1 为什么 CNN 比 MLP 更适合图像分类

1. **参数效率**: CNN 通过权重共享以约 4 倍少的参数 (109K vs 471K) 实现了更好的准确率。
2. **空间感知**: 卷积操作天然处理图像的 2D 结构, 保留了局部空间关系, 而 MLP 通过展平操作破坏了这些关系。
3. **特征层次**: CNN 学习层次化特征——第一层检测边缘/笔画, 第二层组合为曲线和环形, 全连接层进行最终分类。
4. **平移鲁棒性**: 最大池化提供了固有的平移不变性, 使模型对数字位置的微小偏移具有鲁棒性。

6.2 最有启发性的发现: 权重初始化

🔑 关键洞察

本项目中最有启发性的发现是权重初始化对深层网络可训练性的巨大影响。使用固定的 * 0.01 初始化时, 尽管梯度计算在数学上是正确的, CNN 完全无法学习。切换到 He 初始化后立即恢复了学习能力, 这说明了:

- 仅梯度正确是不够的——梯度的**尺度**同样重要
- 带池化层的深层网络放大了对初始化的敏感性
- He 初始化 ($\sigma = \sqrt{2/fan_in}$) 专为 ReLU 网络设计, 能在各层之间维持梯度幅度

6.3 困难样本分析

CNN 最容易误分类的样本包括:

- 书写风格异常的数字 (如弯曲的带横杠"7"→ 预测为"9")
- 形状相似的数字对 (5/3、6/0、9/4)
- 居中不良或缩放不当的数字
- 笔画缺失或多余的数字

6.4 局限性与未来工作

1. **计算效率**: 我们的手动卷积实现中 col2im 反向传播仍使用循环, 速度慢于完全向量化的实现。完全向量化的 col2im 可进一步提升速度。
2. **数据增强**: 添加旋转、缩放和平移增强可以提高对模糊样本的鲁棒性。
3. **更深的架构**: 探索带批归一化的更深层网络可能将准确率推得更高。
4. **正则化**: 实现 Dropout 可以改善泛化能力, 减少 MLP 训练中观察到的训练-验证差距。

7. 结论

✓ 主要结论

本项目仅使用 NumPy 从零成功实现了自定义神经网络库，在 MNIST 上训练了 MLP 和 CNN 模型，并探索了优化和可视化技术。主要发现：

1. **CNN 显著优于 MLP** (98.91% vs 91.59%)，体现了空间特征提取的重要性。
2. **权重初始化至关重要**——He 初始化使 CNN 能够正常训练，而固定小尺度初始化则完全失败。
3. **Momentum 优化**加速收敛并提升最终准确率 (MLP 上 96.07% vs 90.80%)。
4. **错误分析**表明 CNN 的误分类集中在视觉相似的数字对 (5/3、9/4、6/0)，且大多数错误涉及真正模糊的手写体。
5. **卷积核可视化**证实 CNN 学习了层次化的可解释特征——从第一层的简单边缘检测器到第二层的复杂模式组合。

参考文献

1. 课程讲义：神经网络与深度学习
2. MNIST 数据集：<http://yann.lecun.com/exdb/mnist/>
3. NumPy 文档：<https://numpy.org/doc/>
4. He, K. et al. (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." ICCV.

备注

本报告为神经网络与深度学习课程 Project 1 的实验报告。所有核心操作均使用 NumPy 从零实现，未使用 PyTorch、TensorFlow 等高级深度学习框架。

#实验报告 #深度学习 #课程作业